

Anil Seth



From LOGO to Scratch

Many states like Goa are trying to empower children in schools by teaching them computing. The author has become involved with a school's hobby club, and this article lists a few programming environments he feels school students should be familiar with.

I was apprehensive when I volunteered for a hobby programming session, and when it was suggested that Scratch would be the ideal environment to work in. I had never used MIT Scratch and am more comfortable writing code rather than dragging and dropping graphical tiles to create code. I was hoping to drive the students towards Python's Turtle module. However, Scratch was the environment that appealed to the students. After all, animation is far more exciting!

In the process, what we discovered was that different environments make it easier to explore different programming concepts. The following is a short list of programming environments students could explore. In fact, they should explore each one of them, as that will ensure that they have familiarised themselves with the concepts rather than the syntax. Moving to 'real' programming using languages like C++, Java, etc, should then be far easier.

TurtleArt

TurtleArt is a part of the Sugar desktop environment, better known as the software used on OLPC (one laptop per child). This can be installed and used with any Linux desktop environment as well. For example, on Fedora 24, the installation (and correction for any inconsistency in directory names) is as follows:

```
$ sudo dnf install sugar-turtleart
$ cd /usr/share/sugar/activities
$ sudo ln -s TurtleBlocks.activity TurtleArt.activity
$ cp TurtleArt.activity/turtleblocks.desktop ~/Desktop/
```

Now, clicking on the TurtleBlocks icon should start the TurtleArt development environment.

TurtleArt is an implementation of the LOGO programming language, with extensions, but uses the concept of pluggable blocks from Scratch.

You may think of a turtle as an object that can move, turn and carry a pen which may be moved up or down. And, of course, the colour of the pen can be changed. As soon as the turtle has to create anything more than simple lines, it is obvious that the concept of a function becomes very important.

The example in Figure 1 illustrates the code for drawing squares, with the starting position (0,0) of the

turtle as the centre of the squares. There are no parameters to functions in TurtleArt. A student stores values in a box and retrieves them from the box when needed.

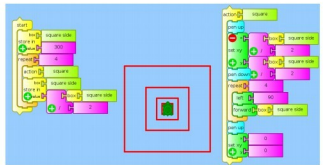


Figure 1: TurtleArt script

Python Turtle

The turtle module in Python is a part of the Tkinter package. The LOGO commands are closely modelled following the Python syntax. A turtle is just a Python class. You may create and manage turtle objects using the full scope of Python.

In particular, it is easy to create multiple turtle objects and control the movement of each. The simple example below illustrates a race between 10 turtles, where their movement is controlled by a random value:

```
from turtle import *
import random

def create_turtle(y):
    turtle = Turtle()
    turtle.penup()
    turtle.goto(0,y)
    turtle.pendown()

# customize the pen of each turtle
turtle.pensize(width=5)
turtle.pencolor((random.random(), random.random(), random.random()))

return turtle

# create a new turtle and position it at different heights.
turtles = []

for number in range(10):
    turtles.append(create_turtle(20*number - 100))
```